

Rede Local em Anel com Passagem de Token sobre UDP

Gabriel Rebello

Gustavo da Silva

Igor Wolwacz

Fundamentos de Redes de Computadores
Junho de 2026

1 Introdução

O trabalho implementa uma rede local em topologia de anel com passagem de token sobre UDP (porta 6000). Foi feito em Node.js, sem dependências externas, usando apenas o módulo nativo `dgram` para a comunicação. O broadcast é usado para os pacotes de controle (`DISCOVER/HELLO`) e unicast para token e dados. O código está dividido em `protocol.js` (montagem e parsing dos pacotes), `crc32.js`, `netinfo.js` (detecção de IP) e `RingMachine.js` (estado do anel, token, fila e timers).

2 Estruturas de dados

As principais estruturas ficam na classe `RingMachine`:

- `hosts` (Map): apelido $\rightarrow$ {ip, ultimoHello}. Base para ordenar o anel, achar o sucessor, resolver IPs e detectar hosts inativos.
- `filaSaida` (Array): fila FIFO de até 10 mensagens. Cada item guarda destino, mensagem e o número de sequência. Só sai da fila após confirmação.
- `seqEsperado` (Map): próximo seq esperado por origem, usado pelo destino para detectar duplicatas.
- Flags escalares: `sucessor`, `souControladora`, `temToken`, `proximoSeqEnvio` e `_ultimaPassagemToken`.
- `CRC_TABLE` (Uint32Array de 256 entradas): tabela pré-computada do CRC32, calculada uma vez na carga do módulo.

3 Concorrência

Node.js é single-threaded e roda em cima de um event loop, então não há threads, locks nem mutexes. A concorrência entre chegada de pacotes, timers (heartbeat, timeout do token, verificação de hosts) e leitura do `stdin` é resolvida pelo próprio loop, que executa um callback por vez até o fim. Isso elimina condições de corrida sobre `hosts`, `filaSaida` e `temToken`. O custo é que nada pode bloquear o loop, por isso toda I/O é assíncrona.

A flag `temToken` funciona como exclusão mútua cooperativa: antes de usar o token o código checa `if (!this.temToken) return;`, e a flag vira `false` antes de qualquer envio, garantindo que o token não seja usado duas vezes.

Tipo	Código	Transporte
DISCOVER	10	broadcast
HELLO	20	broadcast
TOKEN	1000	unicast
DADOS	2000	unicast

4 Formato dos pacotes

- DISCOVER: 10:<apelido>:<ip>
- HELLO: 20:<apelido>:<ip>:<crc32> (CRC sobre o campo vazio, terminado em :). CRC inválido → descarte silencioso.
- TOKEN: a string literal 1000. Qualquer pacote 1000 com campos extras é descartado no primeiro salto (protege contra a corrupção 2000 → 1000), e esse descarte dispara o timeout na controladora.
- DADOS: 2000:<origem>:<destino>:<flag>:<seq>:<ttl>:<msg>:<crc32>. Flag pode ser `maquinainexistente`, `ACK` ou `NAK`.

5 CRC32

Usamos o CRC32/ISO-HDLC (CRC32b), igual ao `binascii.crc32` do Python e ao `zlib`: polinômio `0xEDB88320` refletido, seed `0xFFFFFFFF`, XOR final `0xFFFFFFFF`. Implementação própria por tabela, sem usar `zlib`:

```
function crc32(data) {
  const buf = Buffer.isBuffer(data) ? data : Buffer.from(String(
    data), 'utf8');
  let crc = 0xFFFFFFFF;
  for (let i = 0; i < buf.length; i++) {
    crc = CRC_TABLE[(crc ^ buf[i]) & 0xFF] ^ (crc >>> 8);
  }
  return (crc ^ 0xFFFFFFFF) >>> 0;
}
```

Validamos byte a byte contra o Python para garantir interoperabilidade com outros grupos. Sempre que um campo muda (flag ou TTL), o CRC é recalculado antes de reenviar.

6 Funcionamento do anel

6.1 Inicialização

Cada máquina faz `bind` na porta 6000 (com `reuseAddr`), envia `DISCOVER` em broadcast e espera 1s coletando `HELLO`s. Depois ordena os apelidos alfabeticamente, define o sucessor (circular) e a máquina de menor apelido gera o token e vira controladora. Sozinha em 1s, espera sem gerar token; sem nenhum `HELLO` em 10s, reenvia o `DISCOVER`.

6.2 Token e dados

Ao receber o token, se a fila estiver vazia repassa para o sucessor; se tiver mensagem, envia o pacote de dados e segura o token até o pacote voltar. A origem usa $TTL = 2 \times$ número de máquinas. Cada salto intermediário confere o CRC, decrementa o TTL, recalcula o CRC e repassa. CRC inválido ou $TTL=0 \rightarrow$ descarta (o que provoca o timeout).

No destino: confere o seq esperado. Se for duplicata, não reprocessa mas responde ACK; se for o esperado, imprime a mensagem e avança o contador. Reseta o TTL, marca ACK e devolve pelo anel. Na origem, ao voltar: ACK/maquinainexistente removem a mensagem da fila; NAK mantém na fila (mesmo seq) para retransmitir na próxima posse.

7 Controle do token

A controladora (menor apelido) cuida de dois casos:

- **Token perdido:** um `setTimeout` é rearmado a cada passagem válida; se estourar (`timeoutToken`), gera um token novo.
- **Token duplicado:** se o intervalo desde a última passagem for menor que `tempoMinimoToken`, o token extra é descartado.

```
this._timeoutTokenTimer = setTimeout(() => {
  this.log('TIMEOUT do token detectado. Gerando novo token.');
```

```
  this._ultimaPassagemToken = Date.now();
  this.temToken = true;
  this._processarPosseToken();
  this._armarTimeoutToken();
}, this.timeoutToken);
```

Quando entra uma máquina de apelido menor, ela assume como controladora e a antiga cancela o timer; tokens duplicados gerados na troca somem pelo mecanismo de tempo mínimo.

8 Inserção de falhas

Antes de enviar, a origem pode corromper a mensagem com probabilidade configurável (`probabilidadeErro`). A corrupção troca um caractere aleatório (evitando :) e o CRC é calculado depois da corrupção, então o receptor detecta o erro como se fosse real.

```
_talvezCorromper(mensagem) {
  if (Math.random() < this.probabilidadeErro) {
    const corrompida = this._corromperString(mensagem);
    this.log('[falha simulada] Mensagem corrompida antes do envio
      .');
    return corrompida;
  }
  return mensagem;
}
```

9 Topologia dinâmica

Entrada: a nova máquina manda DISCOVER; quem recebe responde HELLO e reconstrói o anel. **Saída:** detectada pelo heartbeat. Cada máquina reenvia HELLO a cada 10s, e hosts sem HELLO válido por 30s são removidos e o anel é reconstruído. Não há protocolo explícito de saída.

10 Configuração e execução

```
{
  "apelido": "B",
  "ip": "192.168.0.10",
  "broadcastAddr": "255.255.255.255",
  "porta": 6000,
  "delayToken": 1000,
  "delayDados": 500,
  "probabilidadeErro": 0.1,
  "timeoutToken": 8000,
  "tempoMinimoToken": 500
}
```

O campo `ip` é opcional e força o endereço usado, útil quando a máquina tem adaptadores virtuais (VirtualBox/VMware) e a detecção automática pega a interface errada. Execução: `node main.js config.json` (ou `-debug` para logs detalhados).

11 Exemplo de execução

Trecho de log com `-debug` (filtrando HELLO/DISCOVER), mostrando B enviando dados para A, recebendo ACK e repassando o token:

```
[B] [RX] <-- 192.168.0.26:6000    TOKEN    1000
[B] [DEBUG] Token recebido.
[B] [falha simulada] Mensagem corrompida antes do envio.
[B] Enviando DADOS seq=0 destino=A via A.
[B] [TX] --> 192.168.0.26:6000    DADOS    2000:B:A:maquinainexistente
:0:4:ola:...
[B] [RX] <-- 192.168.0.26:6000    DADOS    2000:B:A:ACK:0:4:ola:...
[REDE] B -> A | flag=ACK seq=0 ttl=4 | CRC ok | "ola"
[B] ACK recebido para seq=0 (destino=A). Mensagem entregue.
[B] [DEBUG] Token repassado para A (192.168.0.26).
```